

APIOBike

APIOBike là một dịch vụ chia sẻ xe đạp mới được ra mắt tại thành phố APIO. Một vài trạm được lắp đặt khắp thành phố, giúp cho người dùng có thể mượn hoặc trả xe ở bất kỳ trạm nào. Bởi sự tiện dụng và chi phí thấp, nó đã nhanh chóng trở thành phương tiện di chuyển quan trọng nhất trong thành phố APIO. Tuy nhiên, APIOBike đã gặp phải một vấn đề thường gặp của tất cả các hãng dịch vụ chia sẻ xe: sự mất cân bằng giữa tỷ lệ mượn và trả ở những trạm khác nhau. Kết quả là một số trạm có ít xe đạp, làm cho cư dân xung quanh không thể mượn xe; một số trạm lại có quá nhiều xe, những xe thừa mà người dân xung quanh không cần.

Để giải quyết vấn đề này, APIOBike lên kế hoạch điều động một xe tải tái cân bằng mỗi đêm để phân bổ lại xe đạp giữa các trạm, đảm bảo mỗi trạm đều duy trì số lượng xe đạp phù hợp. Thành phố APIO có toàn bộ N trạm, đánh số $0, 1, \dots, N - 1$. Thông qua quan sát, APIOBike nhận ra rằng mỗi tối, số lượng xe đạp ở trạm i luôn có giá trị $A[i]$, và không ai mượn hoặc trả xe vào buổi tối. Công ty muốn có đúng $B[i]$ xe ở trạm i vào mỗi sáng.

Giữa N trạm, có $N - 1$ con đường được dành cho xe tải tái cân bằng. Mỗi con đường nối giữa hai trạm khác nhau, và xe tải chỉ sử dụng những con đường này để di chuyển. Mỗi con đường có độ dài là một đơn vị. Mạng lưới trạm là liên thông, đảm bảo rằng xe tải có thể di chuyển giữa bất kỳ cặp trạm nào. Mỗi đêm, APIOBike phải điều động một xe tải tái cân bằng để đảm bảo trạm i có đúng $B[i]$ xe. Xe tải này có thể bắt đầu ở bất kỳ trạm nào và kết thúc đường đi của nó ở bất kỳ trạm nào.

Xe tải này là rỗng khi bắt đầu quá trình cân bằng. Bất cứ khi nào xe tải ở tại một trạm, người lái xe có thể bốc bất kỳ số lượng xe đạp nào từ trạm lên xe tải, hoặc dỡ bất kỳ số lượng xe đạp nào từ xe tải xuống trạm. Xe tải và các trạm không có giới hạn sức chứa cho xe đạp nhưng số lượng xe đạp ở bất kỳ địa điểm nào phải không bao giờ xuống dưới không. Công ty muốn xác định khoảng cách di chuyển tối thiểu có thể để hoàn thành việc tái cân bằng và chiến lược tương ứng.

Chi tiết cài đặt

Bạn cần xây dựng hàm sau.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : một mảng có độ dài N , trong đó $A[i]$ là số lượng xe đạp tại trạm i mỗi buổi tối.
- B : một mảng có độ dài N , trong đó $B[i]$ là số lượng xe đạp mục tiêu tại trạm i mỗi sáng.
- U, V : các mảng có độ dài $N - 1$ biểu thị mạng lưới đường bộ. Với mỗi $0 \leq i < N - 1$, con đường thứ i kết nối trạm $U[i]$ và trạm $V[i]$.
- Hàm này được gọi **tối đa 150 000 lần** với mỗi trường hợp thử nghiệm.

Hàm này cần trả về một cặp mảng (X, Y) có độ dài bằng nhau là $k + 1$, biểu thị một chiến lược tái cân bằng:

- X : chuỗi chỉ số các trạm được ghé thăm theo thứ tự thời gian.
- Y : tổng số xe được vận chuyển ở mỗi trạm ghé thăm. Với mỗi $0 \leq j \leq k$:
 - Nếu $Y[j] \geq 0$, tài xế dỡ xuống $Y[j]$ xe đạp ở trạm $X[j]$ từ xe tải, tăng số lượng xe của trạm lên $Y[j]$.
 - Nếu $Y[j] < 0$, tài xế bốc lên $-Y[j]$ xe đạp từ trạm $X[j]$ lên xe tải, giảm số lượng xe của trạm đi $-Y[j]$.

Một chiến thuật tái cân bằng **hợp lệ** (X, Y) với khoảng cách di chuyển k phải thỏa mãn những điều kiện sau:

- Với mỗi $0 \leq j \leq k$, $0 \leq X[j] < N$.
- Với mỗi $0 \leq j < k$, trạm $X[j]$ và trạm $X[j + 1]$ được nối bởi một con đường.
- Với mỗi $0 \leq j \leq k$, $\sum_{t=0}^j Y[t] \leq 0$.
- Với mỗi trạm $0 \leq i < N$ và mỗi bước $0 \leq j \leq k$, gọi $S_{i,j}$ là tổng của $Y[t]$ qua tất cả các bước t mà $0 \leq t \leq j$ và $X[t] = i$.
 - Số lượng xe tại trạm i không bao giờ xuống dưới không: $A[i] + S_{i,j} \geq 0$.
 - Sau khi chiến thuật được hoàn thành, mỗi trạm i phải có đúng $B[i]$ xe đạp: $A[i] + S_{i,k} = B[i]$.

Ràng buộc

- $2 \leq N \leq 300\,000$
- Tổng các N qua tất cả lời gọi đến hàm `find_rebalancing_strategy` không vượt quá 300 000 với mỗi trường hợp thử nghiệm.
- $0 \leq U[i], V[i] < N$ và $U[i] \neq V[i]$ với mỗi i sao cho $0 \leq i < N$.

- $0 \leq A[i], B[i] \leq 10^9$ với mỗi i sao cho $0 \leq i < N$.
- Có thể di chuyển giữa bất kỳ cặp trạm nào.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- Tồn tại ít nhất một giá trị i sao cho $0 \leq i < N$ và $A[i] \neq B[i]$.

Các subtask và Chấm điểm

Ở đây, T là số lần gọi hàm `find_rebalancing_strategy`, và $\sum N$ là tổng các N qua tất cả lời gọi đến hàm `find_rebalancing_strategy`.

Subtask	Điểm	Ràng buộc bổ sung
1	4	Các trạm và các con đường thỏa mãn tính chất P (xem bên dưới). Tồn tại duy nhất một i sao cho $0 \leq i < N$ và $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. Các trạm và các con đường thỏa mãn tính chất P.
3	16	Các trạm và các con đường thỏa mãn tính chất P.
4	9	Tồn tại duy nhất một i sao cho $0 \leq i < N$ và $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Không có ràng buộc nào thêm.

Tính chất P: Với mỗi $0 \leq i < N - 1$, $U[i] = i$ và $V[i] = i + 1$. Với mỗi $0 \leq i < N$, $A[i] \neq B[i]$.

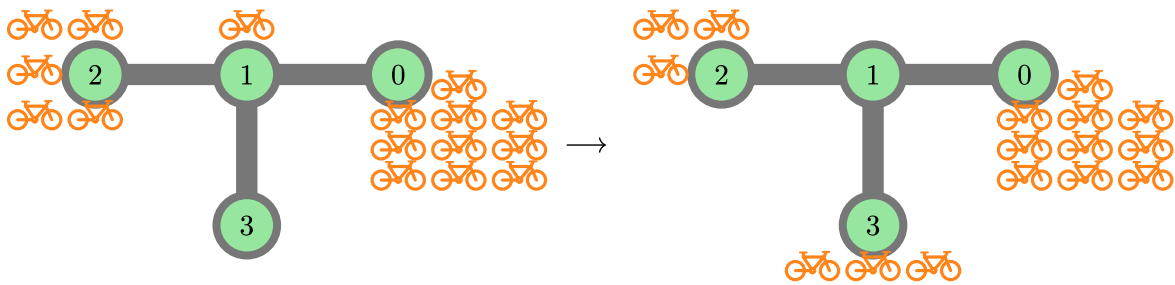
Nếu mảng trả về X và Y có độ dài chính xác là $k^* + 1$, trong đó k^* là khoảng cách di chuyển tối thiểu có thể, nhưng (X, Y) không phải là một chiến lược hợp lệ, bạn sẽ nhận được 50% điểm số.

Ví dụ

Ví dụ 1

Xét lời gọi hàm sau:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



Có $N = 4$ trạm ở thành phố APIO. Ban đầu, các trạm có $A = [10, 1, 5, 0]$ xe, và mục tiêu là có $B = [10, 0, 3, 3]$ xe ở mỗi trạm.

Giả sử xe tải tái cân bằng bắt đầu ở trạm 2, và thực hiện các bước:

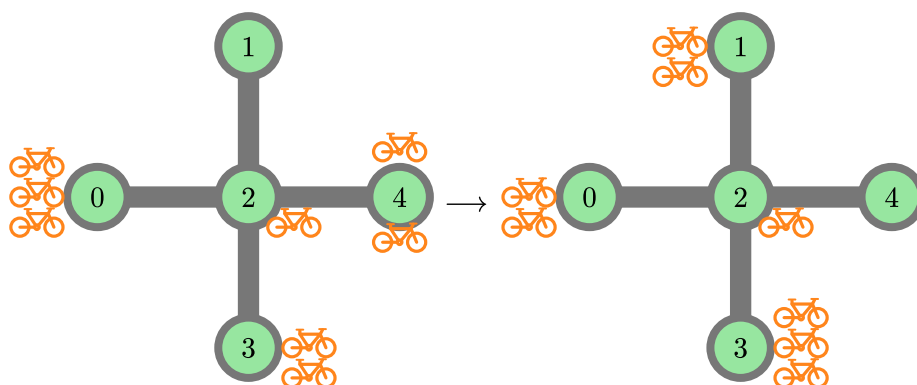
- Ở trạm 2, bốc 2 xe đạp lên xe tải. Trạm 2 giờ có 3 xe đạp, và xe tải có 2 xe đạp.
- Đi tới trạm 1 và bốc 1 xe đạp lên xe tải. Trạm 1 giờ có 0 xe đạp, và xe tải có 3 xe đạp.
- Đi tới trạm 3 và dỡ xuống 3 xe đạp. Trạm 3 giờ có 3 xe đạp, và xe tải có 0 xe đạp.

Tổng khoảng cách di chuyển là 2. Các mảng trả về tương ứng cho chiến lược này là $X = [2, 1, 3]$ và $Y = [-2, -1, 3]$. Có thể chứng minh rằng chiến lược này đạt được khoảng cách tối thiểu. Do đó, hàm này cần trả về $([2, 1, 3], [-2, -1, 3])$.

Ví dụ 2

Xét lời gọi hàm sau:

```
find_rebalancing_strategy(5,
                          [3, 0, 1, 2, 2],
                          [2, 2, 1, 3, 0],
                          [2, 2, 2, 2],
                          [0, 4, 3, 1])
```



Có $N = 5$ trạm ở thành phố APIO. Ban đầu, các trạm có $A = [3, 0, 1, 2, 2]$ xe, và mục tiêu là có $B = [2, 2, 1, 3, 0]$ xe ở mỗi trạm.

Giả sử xe tải tái cân bằng bắt đầu từ trạm 0 và thực hiện các bước sau:

- Ở trạm 0, bốc lên 1 xe.
- Đi tới trạm 2 và bốc lên 1 xe.
- Đi tới trạm 1 và dỡ xuống 2 xe.
- Đi tới trạm 2.
- Đi tới trạm 4 và bốc lên 2 xe.
- Đi tới trạm 2 và dỡ xuống 1 xe.
- Đi tới trạm 3 và dỡ xuống 1 xe.

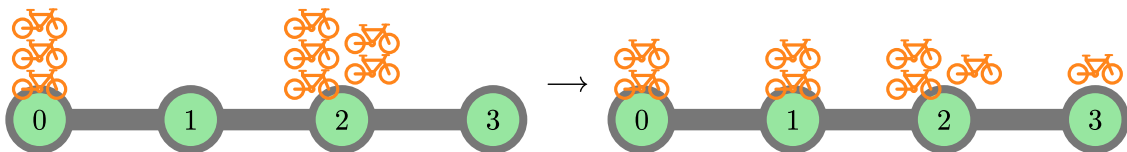
Tổng khoảng cách di chuyển là 6. Các mảng trả về tương ứng cho chiến lược này là $X = [0, 2, 1, 2, 4, 2, 3]$ và $Y = [-1, -1, 2, 0, -2, 1, 1]$. Có thể chứng minh rằng chiến lược này đạt được khoảng cách tối thiểu. Do đó, hàm này cần trả về $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$.

Các chiến thuật hợp lệ khác với khoảng cách 6, chẳng hạn $X = [0, 2, 3, 2, 4, 2, 1]$ và $Y = [-1, 0, 1, 0, -2, 0, 2]$, cũng được xem là đúng đắn. Trả về mảng X, Y với độ dài $7 = 6 + 1$ nhưng không phải chiến thuật hợp lệ, chẳng hạn $X = Y = [0, 0, 0, 0, 0, 0, 0]$, sẽ nhận được 50% số điểm.

Ví dụ 3

Xét lời gọi hàm sau:

```
find_rebalancing_strategy(4,  
                          [3, 0, 5, 0],  
                          [2, 2, 3, 1],  
                          [0, 1, 2],  
                          [1, 2, 3])
```



Một lời giải tối ưu là $X = [2, 1, 0, 1, 2, 3]$ và $Y = [-1, 1, -1, 1, -1, 1]$. Hàm này cần trả về $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. Các chiến thuật hợp lệ khác, chẳng hạn $X = [2, 1, 0, 1, 2, 3]$ và $Y = [-2, 2, -1, 0, 0, 1]$, cũng được xem là đúng đắn.

Ví dụ này thỏa mãn ràng buộc của subtask 2 và 3.

Trình chấm mẫu

Dòng đầu tiên của dữ liệu vào phải chứa một số nguyên duy nhất T , là số lượng kịch bản. Tiếp theo là mô tả về T kịch bản, mỗi kịch bản tuân theo định dạng được chỉ định bên dưới.

Định dạng dữ liệu vào:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Định dạng dữ liệu ra:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```